Module-3

C Compilers and Optimization

3.1 Overview of C Compilers and optimization

Optimization is a program transformation technique, which tries to improve the code by making it consume less resources (i.e., CPU, Memory) and deliver high speed.

In optimization, high-level general programming constructs are replaced by very efficient low-level programming codes. A code optimizing process must follow the three rules given below:

- The output code must not, in any way, change the meaning of the program.
- Optimization should increase the speed of the program and if possible, the program should demand a smaller number of resources.
- Optimization should itself be fast and should not delay the overall compiling process.

Optimizing code takes time and reduces source code readability. Usually, it's only worth optimizing functions that are frequently executed and important for performance. It is recommended to use a performance profiling tool, found in most ARM simulators, to find these frequently executed functions.

C compilers must translate C function literally into assembler so that it works for all possible inputs. In practice, many of the input combinations are not possible or won't occur. Let's start by looking at an example of the problems the compiler faces.

Example: The memclr function clears N bytes of memory at address data.

```
void memclr(char *data, int N)
{
for (; N>0; N--)
{
*data=0; data++;
}
}
```

- No matter how advanced the compiler, it does not know whether N can be 0 on input or not. Therefore, the compiler needs to test for this case explicitly before the first iteration of the loop.
- The compiler doesn't know whether the data array pointer is four-byte aligned or not. If it is four-byte aligned, then the compiler can clear four bytes at a time using an int store rather than a char store.
- Nor does it know whether N is a multiple of four or not. If N is a multiple of four, then the compiler can repeat the loop body four times or store four bytes at a time using an int store.

The compiler must be conservative and assume all possible values for N and all possible alignments for data. To write efficient C code, you must be aware of areas where the C compiler must be conservative, the limits of the processor architecture the C compiler is mapping to, and the limits of a specific C compiler.

VTUADDA

3.2 Basic C Data Types

ARM processors have 32-bit registers and 32-bit data processing operations. The ARM architecture is a RISC load/store architecture. In other words, it is necessary to load values from memory into registers before acting on them. There are no arithmetic or logical instructions that manipulate values in memory directly.

- Early versions of the ARM architecture (ARMv1 to ARMv3) provided hardware support for loading and storing unsigned 8-bit and unsigned or signed 32-bit values. Table 1 shows the load/store instruction classes available by ARM architecture.
- In Table 1 loads that act on 8- or 16-bit values extend the value to 32 bits before writing to an ARM register. **Unsigned values are zero-extended** and **signed values sign-extended**. This means that the **cast of a loaded value to an int type does not cost extra instructions.**
- The ARMv4 architecture and above support signed 8-bit and 16-bit loads and stores directly, through new instructions.
- Finally, ARMv5 adds instruction support for 64-bit load and stores. This is available in ARM9E and later cores.
- Prior to ARMv4, ARM processors were not good at handling signed 8-bit or any 16-bit values. Therefore, **ARM C compilers define char to be an unsigned 8-bit value**, rather than a signed 8-bit value as is typical in many other compilers.

Table 1: Load and store instructions by ARM architecture.

Architecture	Instruction	Action
Pre-ARMv4	LDRB	load an unsigned 8-bit value
	STRB	store a signed or unsigned 8-bit value
	LDR	load a signed or unsigned 32-bit value
	STR	store a signed or unsigned 32-bit value
ARMv4	LDRSB	load a signed 8-bit value
	LDRH	load an unsigned 16-bit value
	LDRSH	load a signed 16-bit value
	STRH	store a signed or unsigned 16-bit value
ARMv5	LDRD	load a signed or unsigned 64-bit value
	STRD	store a signed or unsigned 64-bit value

Table 2: C compiler datatype mappings.

C Data Type	Implementation
`char`	unsigned 8-bit byte
`short`	signed 16-bit halfword
`int`	signed 32-bit word
`long`	signed 32-bit word
`long long`	signed 64-bit double word

3.2.1 Local variable types

- ARMv4-based processors can efficiently load and store 8, 16, and 32-bit data. However, most ARM data processing operations are 32-bit only. For this reason, you should **use a 32-bit datatype, int or long, for local variables** wherever possible.
- **Avoid using char and short as local variable types**, even if you are manipulating an 8- or 16-bit value. The one exception is when you want wrap-around to occur (If you require modulo arithmetic of the form 255+1=0, then use the char type.)

To see the effect of local variable types, let's consider a simple example.

Examples: We'll look in detail at a **checksum function** that sums the values in a data packet. Most communication protocols (such as TCP/IP) have a checksum or cyclic redundancy check (CRC) routine to check for errors in a data packet.

1. The following code checksums a data packet containing 64 words. It shows why you should avoid using char for local variables.

```
int checksum_v1(int *data)
{
char i; int sum = 0;
for (i = 0; i < 64; i++)
{
sum += data[i];
}
return sum;
}
```

VTUADDA

At first sight it looks as though declaring i as a char is efficient. You may be thinking that a char uses less register space or less space on the ARM stack than an int. On the ARM, both these assumptions are wrong.

All ARM registers are 32-bit, and all stack entries are at least 32-bit. Furthermore, to implement the i++ exactly, the compiler must account for the case when i = 255. Any attempt to increment 255 should produce the answer 0.

Consider the compiler output for this function. We've added labels and comments to make the assembly clear.

```
checksum_v1
        MOV     r2,r0           ; r2 = data
        MOV     r0,#0           ; sum = 0
        MOV     r1,#0           ;i=0
checksum_v1_loop
        LDR     r3,[r2,r1,LSL #2]       ; r3 = data[i]
        ADD     r1,r1,#1                ; r1 = i+1
        AND     r1,r1,#0xff             ; i = (char)r1
        CMP     r1,#0x40                ; compare i, 64
        ADD     r0,r3,r0                ; sum += r3
        BCC     checksum_v1_loop        ; if (i<64) loop
        MOV     pc,r14                  ; return sum
```

2. Now compare this to the compiler output where instead we declare **i as an unsigned int**.

```
checksum_v2
        MOV     r2,r0                   ; r2 = data
        MOV     r0,#0                   ; sum = 0
        MOV     r1,#0                   ;i=0
checksum_v2_loop
        LDR     r3,[r2,r1,LSL #2]       ; r3 = data[i]
        ADD     r1,r1,#1                ; r1++
        CMP     r1,#0x40                ; compare i, 64
        ADD     r0,r3,r0                ; sum += r3
        BCC     checksum_v2_loop        ; if (i<64) goto loop
        MOV     pc,r14                  ; return sum
```

In the first case, the compiler inserts an **extra AND instruction to reduce i to the range 0 to 255 before the comparison with 64.** This instruction disappears in the second case.

3. Next, suppose the data packet contains 16-bit values and we need a 16-bit checksum.

```
short checksum_v3(short *data)
{
unsigned int i;
short sum = 0;
for (i = 0; i < 64; i++)
{
sum = (short)(sum + data[i]);
}
return sum;
}
```

You may wonder why the for-loop body doesn't contain the code: sum += data[i];

The expression sum + data[i] is an integer and so can only be assigned to a short using an (implicit or explicit) **narrowing cast.** As you can see in the following assembly output, the compiler must insert extra instructions to implement the narrowing cast:

```
checksum_v3
        MOV     r2,r0                   ; r2 = data
        MOV     r0,#0                   ; sum = 0
        MOV     r1,#0                   ;i=0
checksum_v3_loop
        ADD     r3,r2,r1,LSL #1         ; r3 = &data[i]
        LDRH    r3,[r3,#0]              ; r3 = data[i]
        ADD     r1,r1,#1                ; i++
        CMP     r1,#0x40                ; compare i, 64
        ADD     r0,r3,r0                ; r0 = sum + r3
        MOV     r0,r0,LSL #16
        MOV     r0,r0,ASR #16           ; sum = (short)r0
        BCC     checksum_v3_loop        ; if (i<64) goto loop
        MOV     pc,r14                  ; return sum
```

VTUADDA

The loop is now three instructions longer than the loop for example checksum_v2 earlier. There are two reasons for the extra instructions:

- The LDRH instruction does not allow for a shifted address offset as the LDR instruction did in checksum_v2. Therefore, the first ADD in the loop calculates the address of item i in the array. The LDRH loads from an address with no offset.
- The cast reducing total + array[i] to a short requires two MOV instructions. The compiler shifts left by 16 and then right by 16 to implement a 16-bit sign extend. The shift right is a sign-extending shift, so it replicates the sign bit to fill the upper 16 bits.

We can avoid the second problem by using an int type variable to hold the partial sum. We only reduce the sum to a short type at the function exit.

However, the first problem is a new issue. We can solve it by accessing the array by incrementing the pointer data rather than using an index as in data[i]. This is efficient regardless of array type size or element size. All ARM load and store instructions have a post increment addressing mode.

4. The checksum_v4 code fixes all the problems we have discussed in this section. It uses int type local variables to avoid unnecessary casts. It increments the pointer data instead of using an index offset data[i].

```
short checksum_v4(short *data)
{
unsigned int i;
int sum=0;
for (i=0; i<64; i++)
{
sum += *(data++);
}
return (short)sum;
}
```

VTUADDA

The *(data++) operation translates to a single ARM instruction that loads the data and increments the data pointer. Of course, you could write sum += *data; data++; or even *data++ instead if you prefer.

The compiler produces the following output. Three instructions have been removed from the inside loop, saving three cycles per loop compared to checksum_v3.

```
checksum_v4
        MOV     r2,#0                   ; sum = 0
        MOV     r1,#0                   ;i=0
checksum_v4_loop
        LDRSH   r3,[r0],#2              ; r3 = *(data++)
        ADD     r1,r1,#1                ; i++
        CMP     r1,#0x40                ; compare i, 64
        ADD     r2,r3,r2                ; sum += r3
        BCC     checksum_v4_loop        ; if (sum<64) goto loop
        MOV     r0,r2,LSL #16
```

```
MOV    r0,r0,ASR #16        ; r0 = (short)sum
MOV    pc,r14               ; return r0
```

The compiler is still performing one cast to a 16-bit range, on the function return. You could remove this also by returning an int result.

3.2.2 Function Argument types

We know that converting local variables from types char or short to type int **increases performance and reduces code size.** The same holds for function arguments.

Consider the following simple function, which adds two 16-bit values, halving the second, and returns a 16-bit sum:

```
short add_v1(short a, short b)
{
return a+ (b >> 1);
}
```

This function is useful test case to illustrate the problems faced by the compiler. The input values a, b, and the return value will be passed in 32-bit ARM registers.

Should the compiler assume that these 32-bit values are in the range of a short type, that is, 32,768 to 32,767? Or should the compiler force values to be in this range by sign-extending the lowest 16 bits to fill the 32-bit register? **The compiler must make compatible decisions for the function caller and callee.** Either the caller or callee must perform the cast to a short type.

We say that function arguments are passed **wide** if they are **not reduced to the range** of the type and **narrow** if they are.

If the compiler

- passes arguments wide, then the callee must reduce function arguments to the correct range.
- passes arguments narrow, then the caller must reduce the range.
- returns values wide, then the caller must reduce the return value to the correct range.
- returns values narrow, then the callee must reduce the range before returning the value.

The armcc output for add_v1 shows that the compiler casts the return value to a short type but does not cast the input values. It assumes that the caller has already ensured that the 32-bit values r0 and r1 are in the range of the short type. This shows narrow passing of arguments and return value.

```
add_v1
    ADD    r0,r0,r1,ASR #1      ; r0 = (int)a + ((int)b >> 1)
    MOV    r0,r0,LSL #16
    MOV    r0,r0,ASR #16        ; r0 = (short)r0
    MOV    pc,r14               ; return r0
```

The gcc compiler we used is more cautious and makes no assumptions about the range of argument value. This version of the compiler reduces the input arguments to the range of a

short in both the caller and the callee. It also casts the return value to a short type. Here is the compiled code for add_v1:

```
add_v1_gcc
        MOV     r0, r0, LSL #16
        MOV     r1, r1, LSL #16
        MOV     r1, r1, ASR #17         ; r1 = (int)b >> 1
        ADD     r1, r1, r0, ASR #16     ; r1 += (int)a
        MOV     r1, r1, LSL #16
        MOV     r0, r1, ASR #16         ; r0 = (short)r1
        MOV     pc, lr                  ; return r0
```

Whatever the merits of different narrow and wide calling protocols, you can see that **char or short type function arguments and return values introduce extra casts**. These **increase code size and decrease performance**.

It is more efficient to use the int type for function arguments and return values, even if you are only passing an 8-bit value.

3.2.3 Signed versus Unsigned Types

The previous sections demonstrate the advantages of using int rather than a char or short type for local variables and function arguments. This section compares the efficiencies of signed int and unsigned int.

- If your code uses addition, subtraction, and multiplication, then there is no performance difference between signed and unsigned operations.
- However, there is a difference when it comes to division.

Consider the following short example that averages two integers:

```
int average_v1(int a, int b)
{
return (a+b)/2;
}
```

This compiles to

```
average_v1
        ADD     r0,r0,r1                ; r0=a+b
        ADD     r0,r0,r0,LSR #31        ; if (r0<0) r0++
        MOV     r0,r0,ASR #1            ; r0 = r0 >> 1
        MOV     pc,r14                  ; return r0
```

Notice that the compiler adds one to the sum before shifting by right if the sum is negative. In other words it replaces x/2 by the statement: (x<0) ? ((x+1) >> 1): (x >> 1). It must do this because x is signed.

It is more efficient to use unsigned types for divisions. The compiler converts unsigned power of two divisions directly to right shifts. For general divisions, the divide routine in the C library is faster for unsigned types.

VTUADDA

SUMMARY: The Efficient Use of C Types

- For local variables held in registers, don't use a char or short type unless 8-bit or 16-bit modular arithmetic is necessary. Use the signed or unsigned int types instead. Unsigned types are faster when you use divisions.
- For array entries and global variables held in main memory, use the type with the smallest size possible to hold the required data. This saves memory footprint.
  The ARMv4 architecture is efficient at loading and storing all data widths provided you traverse arrays by incrementing the array pointer. Avoid using offsets from the base of the array with short type arrays, as LDRH does not support this.
- Use explicit casts when reading array entries or global variables into local variables or writing local variables out to array entries.
  The casts make it clear that for fast operation you are taking a narrow width type stored in memory and expanding it to a wider type in the registers. Switch on implicit narrowing cast warnings in the compiler to detect implicit casts.
- Avoid implicit or explicit narrowing casts in expressions because they usually cost extra cycles. Casts on loads or stores are usually free because the load or store instruction performs the cast for you.
- Avoid char and short types for function arguments or return values. Instead use the int type even if the range of the parameter is smaller. This prevents the compiler performing unnecessary casts.

3.3 C Looping Structures

3.3.1 Loops with a fixed number of iterations

What is the most efficient way to write a for loop on the ARM? Let's return to our checksum example and look at the looping structure. Here is the last version of the 64-word packet checksum routine. This shows how the compiler treats a loop with incrementing count i++.

```
int checksum_v5(int *data)
{
unsigned int i; int sum=0;
for (i=0; i<64; i++)
{
sum += *(data++);
}
return sum;
}
```

This compiles to

```
checksum_v5
    MOV    r2,r0    ; r2 = data
    MOV    r0,#0    ; sum = 0
    MOV    r1,#0    ;i=0
checksum_v5_loop
    LDR    r3,[r2],#4      ; r3 = *(data++)
    ADD    r1,r1,#1        ; i++
    CMP    r1,#0x40        ; compare i, 64
```

```
ADD     r0,r3,r0            ; sum += r3
BCC     checksum_v5_loop    ; if (i<64) goto loop
MOV     pc,r14              ; return sum
```

It takes three instructions to implement the for-loop structure:

- An ADD to increment i
- A compare to check if i is less than 64
- A conditional branch to continue the loop if i < 64

This is not efficient. On the ARM, a loop should only use two instructions:

- A **subtract** to decrement the loop counter, which **also sets the condition code flags** on the result.
- A **conditional branch instruction**

The key point is that the loop counter should count down to zero rather than counting up to some arbitrary limit. Then the comparison with zero is free since the result is stored in the condition flags.

Example:

1. shows the improvement if we switch to a decrementing loop rather than an incrementing loop.

```
int checksum_v6(int *data)
{
unsigned int i; int sum=0;
for (i=64; i!=0; i--)
{
sum += *(data++);
}
return sum;
}
```

This compiles to

checksum_v6

```
    MOV     r2,r0               ; r2 = data
    MOV     r0,#0               ; sum = 0
    MOV     r1,#0x40            ;i= 64
checksum_v6_loop
    LDR     r3,[r2],#4          ; r3 = *(data++)
    SUBS    r1,r1,#1            ; i-- and set flags
    ADD     r0,r3,r0            ; sum += r3
    BNE     checksum_v6_loop    ; if (i!=0) goto loop
    MOV     pc,r14              ; return sum
```

VTUADDA

The SUBS and BNE instructions implement the loop. Our checksum example now has the minimum number of four instructions per loop. This is much better than six for checksum_v1 and eight for checksum_v3.

For an unsigned loop counter i we can use either of the loop continuation conditions i!=0 or i>0. As i can't be negative, they are the same condition.

For a signed loop counter, it is tempting to use the condition i>0 to continue the loop. You might expect the compiler to generate the following two instructions to implement the loop:

```
SUBS    r1,r1,#1        ; compare i with 1, i=i-1
BGT     loop            ; if (i+1>1) goto loop
```

In fact, the compiler will generate,

```
SUB     r1,r1,#1        ; i--
CMP     r1,#0           ; compare i with 0
BGT     loop            ; if (i>0) goto loop
```

Therefore you should **use the termination condition i!=0 for signed or unsigned loop counters**. It saves one instruction over the condition i>0 for signed i.

3.3.2 Loops using a variable number of iterations.

Now suppose we want our checksum routine to handle packets of arbitrary size. We pass in a variable N giving the number of words in the data packet. Using the lessons from the last section we count down until N=0 and don't require an extra loop counter i.

Example: 1. The checksum_v7 shows how the compiler handles a for loop with a variable number of iterations N.

```
int checksum_v7(int *data, unsigned int N)
{
int sum=0;
for (; N!=0; N--)
{
sum += *(data++);
}
return sum;
}
```

This compiles to

```
checksum_v7
    MOV     r2,#0               ; sum = 0
    CMP     r1,#0               ; compare N, 0
    BEQ     checksum_v7_end     ; if (N==0) goto end
checksum_v7_loop
    LDR     r3,[r0],#4          ; r3 = *(data++)
    SUBS    r1,r1,#1            ; N-- and set flags
    ADD     r2,r3,r2            ; sum += r3
    BNE     checksum_v7_loop    ; if (N!=0) goto loop
```


VTUADDA

```
checksum_v7_end
        MOV     r0,r2                   ; r0 = sum
        MOV     pc,r14                  ; return r0
```

Notice that the compiler checks that N is nonzero on entry to the function. Often this check is unnecessary since you know that the array won't be empty. In this case a do-while loop gives better performance and code density than a for loop.

2. This example shows how to use a do-while loop to remove the test for N being zero that occurs in a for loop.

```
int checksum_v8(int *data, unsigned int N)
{
int sum=0;
do
{
sum += *(data++);
} while (--N!=0); return sum;
}
```

The compiler output is now

```
checksum_v8
        MOV     r2,#0                   ; sum = 0
checksum_v8_loop
        LDR     r3,[r0],#4              ; r3 = *(data++)
        SUBS    r1,r1,#1                ; N-- and set flags
        ADD     r2,r3,r2                ; sum += r3
        BNE     checksum_v8_loop        ; if (N!=0) goto loop
        MOV     r0,r2                   ; r0 = sum
        MOV     pc,r14                  ; return r0
```

VTUADDA

Compare this with the output for checksum_v7 to see the two-cycle saving.

3.3.3 Loop Unrolling

We saw in Section 3.3.1 that each loop iteration costs two instructions in addition to the body of the loop: a subtract to decrement the loop count and a conditional branch. We call these instructions the **loop overhead.**

On ARM7 or ARM9 processors the subtract takes one cycle and the branch three cycles, giving an overhead of four cycles per loop.

You can **save some of these cycles** by **unrolling a loop - repeating the loop body several times and reducing the number of loop iterations by the same proportion**.

Example,

1. let's unroll our packet checksum example **four times**.

The following code unrolls our packet checksum loop by four times. We assume that the number of words in the packet N is a multiple of four.

```
int checksum_v9(int *data, unsigned int N)
{
int sum=0;
do
{
sum += *(data++);
sum += *(data++);
sum += *(data++);
sum += *(data++);
N -= 4;
} while ( N!=0);
return sum;
}
```

This compiles to

```
checksum_v9
        MOV r2,#0                   ; sum = 0
checksum_v9_loop
        LDR     r3,[r0],#4          ; r3 = *(data++)
        SUBS    r1,r1,#4            ; N -= 4 & set flags
        ADD     r2,r3,r2            ;sum += r3
        LDR     r3,[r0],#4          ;r3 = *(data++)
        ADD     r2,r3,r2            ;sum += r3
        LDR     r3,[r0],#4          ;r3 = *(data++)
        ADD     r2,r3,r2            ;sum += r3
        LDR     r3,[r0],#4          ;r3 = *(data++)
        ADD     r2,r3,r2            ;sum += r3
        BNE     checksum_v9_loop    ;if (N!=0) goto loop
        MOV     r0,r2               ;r0 = sum
        MOV     pc,r14              ;return r0
```

VTUADDA

loop overhead has been reduced the from 4N cycles to (4N)/4=N cycles.

On the ARM7TDMI, this accelerates the loop from 8 cycles per accumulate to 20/4=5 cycles per accumulate, nearly doubling the speed!

There are **two questions** you need to ask when unrolling a loop:

- How many times should I unroll the loop?
- What if the number of loop iterations is not a multiple of the unroll amount? For example, what if N is not a multiple of four in checksum_v9?

To start with the first question, **only unroll loops that are important for the overall performance of the application**. Otherwise unrolling will increase the code size with little performance benefit. Unrolling may even reduce performance by evicting more important code from the cache.

For the second question**, try to arrange it so that array sizes are multiples of unroll amount.** If this isn't possible, then you must add extra code to take care of the leftover cases. This increases the code size a little but keeps the performance high.

2. This example handles the checksum of any size of data packet using a loop that has been unrolled four times.

```
int checksum_v10(int *data, unsigned int N)
{
unsigned int i; int sum=0;
for (i=N/4; i!=0; i--)
{
sum += *(data++);
sum += *(data++);
sum += *(data++);
sum += *(data++);
}
for (i=N&3; i!=0; i--)
{
sum += *(data++);
}
return sum;
}
```

The second for loop handles the remaining cases when N is not a multiple of four. Note that both N/4 and N&3 can be zero, so we can't use do-while loops.

SUMMARY: Writing Loops Efficiently

- Use loops that count down to zero. Then the compiler does not need to allocate a register to hold the termination value, and the comparison with zero is free.
- Use unsigned loop counters by default and the continuation condition i!=0 rather than i>0. This will ensure that the loop overhead is only two instructions.
- Use do-while loops rather than for loops when you know the loop will iterate at least once. This saves the compiler checking to see if the loop count is zero.
- Unroll important loops to reduce the loop overhead. Do not over unroll. If the loop overhead is small as a proportion of the total, then unrolling will increase code size and hurt the performance of the cache.
- Try to arrange that the number of elements in arrays are multiples of four or eight. You can then unroll loops easily by two, four, or eight times without worrying about the leftover array elements.

3.4 Register Allocation

The compiler attempts to allocate a processor register to each local variable you use in a C function. It will try to use the same register for different local variables if the use of the variables does not overlap.

When there are more local variables than available registers, the compiler stores the excess variables on the processor stack. These variables are called **spilled or swapped out variables**

VTUADDA

since they are written out to memory. Spilled variables are slow to access compared to variables allocated to registers.

To implement a function efficiently, need to

- minimize the number of spilled variables
- ensure that the most important and frequently accessed variables are stored in registers

First let's look at the number of processor registers the ARM C compilers have available for allocating variables. Table 3 shows the standard register names and usage when following the ARM-Thumb procedure call standard (ATPCS), which is used in code generated by C compilers.

Table 3: C compiler register usage.

Register number	Alternate register names	ATPCS register usage
r0	*a1*	Argument registers. These hold the first four function arguments on a function call and the return value on a function return. A function may corrupt these registers and use them as general scratch registers within the function.
r1	*a2*	
r2	*a3*	
r3	*a4*	
r4	*v1*	General variable registers. The function must preserve the callee values of these registers.
r5	*v2*	
r6	*v3*	
r7	*v4*	
r8	*v5*	
r9	*v6 sb*	General variable register. The function must preserve the callee value of this register except when compiling for *read-write position independence* (RWPI). Then *r9* holds the *static base* address. This is the address of the read-write data.
r10	*v7 sl*	General variable register. The function must preserve the callee value of this register except when compiling with stack limit checking. Then *r10* holds the stack limit address.
r11	*v8 fp*	General variable register. The function must preserve the callee value of this register except when compiling using a frame pointer. Only old versions of *armcc* use a frame pointer.
r12	*ip*	A general scratch register that the function can corrupt. It is useful as a scratch register for function veneers or other intraprocedure call requirements.
r13	*sp*	The stack pointer, pointing to the full descending stack.
r14	*lr*	The link register. On a function call this holds the return address.
r15	*pc*	The program counter.

VTUADDA

In theory, the C compiler can assign 14 variables to registers without spillage. In practice, some compilers use a fixed register such as r12 for intermediate scratch working and do not assign variables to this register. Also, complex expressions require intermediate working registers to evaluate. Therefore, to ensure good assignment to registers, you should try to **limit the internal loop of functions to using at most 12 local variables.**

If the compiler does need to swap out variables, then it **chooses which variables to swap out based on frequency of use**. A variable used inside a loop counts multiple times. You can guide the compiler as to which variables are important by ensuring these variables are used within the innermost loop.

The **register** keyword in C hints that a compiler should allocate the given variable to a register. However, different compilers treat this keyword in different ways, and different architectures have a different number of available registers (for example, Thumb and ARM). Therefore, we recommend that you **avoid using register and rely on the compiler's normal register allocation routine.**

SUMMARY: Efficient Register Allocation

- Try to limit the number of local variables in the internal loop of functions to 12. The compiler should be able to allocate these to ARM registers.
- You can guide the compiler as to which variables are important by ensuring these variables are used within the innermost loop.

3.5 Function calls

The ARM Procedure Call Standard (APCS) defines how to pass function arguments and return values in ARM registers.

The more recent ARM-Thumb Procedure Call Standard (ATPCS) covers ARM and Thumb interworking as well.

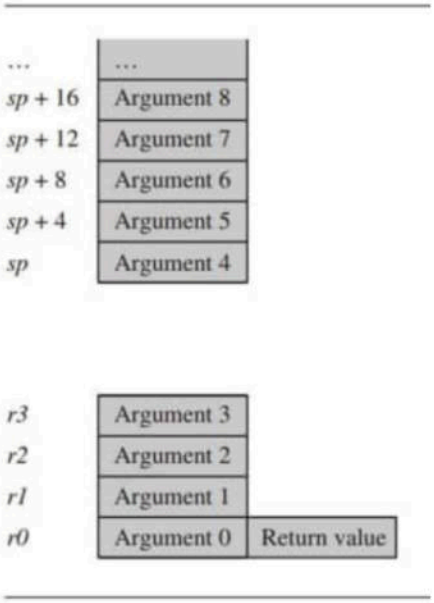
The first four integer arguments are passed in the first four ARM registers: r0, r1, r2, and r3. Subsequent integer arguments are placed on the **full descending stack, ascending in memory** as in Figure below. Function return integer values are passed in r0.

Two-word arguments such as long long or double are passed in a pair of consecutive argument registers and returned in r0, r1.

The first point to note about the procedure call standard is the **four-register rule**. Functions with four or fewer arguments are far more efficient to call than functions with five or more arguments.

- **For functions with four or fewer arguments, the compiler can pass all the arguments in registers.**
- **For functions with more arguments, both the caller and callee must access the stack for some arguments.**

VTUADDA



ATPCS argument passing.

If your C function needs more than four arguments, then it is almost always more efficient to **use structures**. Group related arguments into structures and pass a structure pointer rather than multiple arguments.

Example: 1. illustrates the benefits of using a structure pointer. First, we show a typical routine to insert N bytes from array data into a queue. We implement the queue using a cyclic buffer with start address Q_start (inclusive) and end address Q_end (exclusive).

```
char *queue_bytes_v1(
char *Q_start,          /* Queue buffer start address */
char *Q_end,            /* Queue buffer end address */
char *Q_ptr,            /* Current queue pointer position */
char *data,             /* Data to insert into the queue */
unsigned int N)  /* Number of bytes to insert */
{
do
{
*(Q_ptr++) = *(data++);
if (Q_ptr == Q_end)
{
Q_ptr = Q_start;
}
} while (--N);
return Q_ptr;
}
```

This compiles to

```
queue_bytes_v1
    STR     r14,[r13,#-4]!          ;       save lr on the stack
    LDR     r12,[r13,#4]            ;       r12 = N
queue_v1_loop
```

```
LDRB    r14,[r3],#1          ;       r14 = *(data++)
STRB    r14,[r2],#1          ;       *(Q_ptr++) = r14
CMP     r2,r1                ;       if (Q_ptr == Q_end)
MOVEQ r2,r0                  ;       {Q_ptr = Q_start;}
SUBS    r12,r12,#1           ;       --N and set flags
BNE     queue_v1_loop        ;       if (N!=0) goto loop
MOV     r0,r2                ;       r0 = Q_ptr
LDR     pc,[r13],#4          ;       return r0
```

Compare this with a more structured approach using three function arguments.

2. The following code creates a Queue structure and passes this to the function to reduce the number of function arguments.

```
typedef struct {
char *Q_start;          /* Queue buffer start address */
char *Q_end;            /* Queue buffer end address */
char *Q_ptr;            /* Current queue pointer position */
} Queue;
void queue_bytes_v2(Queue *queue, char *data, unsigned int N)
{
char *Q_ptr = queue->Q_ptr; char *Q_end = queue->Q_end;
do
{
*(Q_ptr++) = *(data++);
if (Q_ptr == Q_end)
{
Q_ptr = queue->Q_start;
}
} while (--N);
queue->Q_ptr = Q_ptr;
}
```

This compiles to

```
queue_bytes_v2
    STR     r14,[r13,#-4]!       ;       save lr on the stack
    LDR     r3,[r0,#8]           ;       r3 = queue->Q_ptr
    LDR     r14,[r0,#4]          ;       r14 = queue->Q_end
queue_v2_loop
    LDRB    r12,[r1],#1          ;       r12 = *(data++)
    STRB    r12,[r3],#1          ;       *(Q_ptr++) = r12
    CMP     r3,r14               ;       if (Q_ptr == Q_end)
    LDREQ r3,[r0,#0]             ;       Q_ptr = queue->Q_start
    SUBS    r2,r2,#1             ;       --N and set flags
    BNE     queue_v2_loop        ;       if (N!=0) goto loop
    STR     r3,[r0,#8]           ;       queue->Q_ptr = r3
    LDR     pc,[r13],#4          ;       return
```

VTUADDA

The queue_bytes_v2 is one instruction longer than queue_bytes_v1, but it is in fact more efficient overall.

The second version has only three function arguments rather than five. Each call to the function requires only three register setups. This compares with four register setups, a stack push, and a stack pull for the first version. **There is a net saving of two instructions in function call overhead**.

There are other ways of reducing function call overhead if your function is very small and corrupts few registers (uses few local variables). **Put the C function in the same C file as the functions that will call it.** The C compiler then knows the code generated for the callee function and can make optimizations in the caller function:

- If the callee function is very small, then the compiler can inline the code in the caller function. This removes the function call overhead completely.

3. The function uint_to_hex converts a 32-bit unsigned integer into an array of eight hexa-decimal digits. It uses a helper function nybble_to_hex, which converts a digit d in the range 0 to 15 to a hexadecimal digit.

```
unsigned int nybble_to_hex(unsigned int d)
{
if (d<10)
{
return d + '0';
}
returnd- 10+ 'A';
}
void uint_to_hex(char *out, unsigned int in)
{
unsigned int i;
for (i=8; i!=0; i--)
{
in = (in << 4) | (in >> 28); /* rotate in left by 4 bits */
*(out++) = (char)nybble_to_hex(in & 15);
}
}
```

When we compile this, we see that uint_to_hex doesn't call nybble_to_hex at all! In the following compiled code, the compiler has inlined the uint_to_hex code. This is more efficient than generating a function call.

```
uint_to_hex
    MOV     r3,#8   ;i=8
uint_to_hex_loop
    MOV     r1,r1,ROR #28       ; in = (in << 4)|(in >> 28)
    AND     r2,r1,#0xf          ; r2 = in & 15
    CMP     r2,#0xa             ; if (r2>=10)
    ADDCS r2,r2,#0x37           ; r2 +='A'-10
    ADDCC r2,r2,#0x30           ; else r2 +='0'
```

```
STRB    r2,[r0],#1          ; *(out++) = r2
SUBS    r3,r3,#1            ; i-- and set flags
BNE     uint_to_hex_loop    ; if (i!=0) goto loop
MOV     pc,r14              ; return
```

The compiler will only **inline small functions**. You can ask the compiler to inline a function using the **__inline keyword**. Inlining large functions can lead to big increases in code size without much performance improvement.

SUMMARY: Calling Functions Efficiently

- Try to restrict functions to four arguments. This will make them more efficient to call. Use structures to group related arguments and pass structure pointers instead of multiple arguments.
- Define small functions in the same source file and before the functions that call them. The compiler can then optimize the function call or inline the small function.
- Critical functions can be inlined using the __inline keyword.

3.6 Pointer Aliasing

Two pointers are said to alias when they **point to the same address**. If you write to one pointer, it will affect the value you read from the other pointer.

In a function, the compiler often doesn't know which pointers can alias and which pointers can't. The compiler must be very pessimistic and assume that any write to a pointer may affect the value read from any other pointer, which can significantly reduce code efficiency.

Example: The following function increments two timer values by a step amount:

```
void timers_v1(int *timer1, int *timer2, int *step)
{
*timer1 += *step;
*timer2 += *step;
}
```

This compiles to

```
timers_v1
    LDR     r3,[r0,#0]      ; r3 = *timer1
    LDR     r12,[r2,#0]     ; r12 = *step
    ADD     r3,r3,r12       ; r3 += r12
    STR     r3,[r0,#0]      ; *timer1 = r3
    LDR     r0,[r1,#0]      ; r0 = *timer2
    LDR     r2,[r2,#0]      ; r2 = *step
    ADD     r0,r0,r2; r0 += r2
    STR     r0,[r1,#0]      ; *timer2 = t0
    MOV     pc,r14  ; return
```

Note that the compiler loads from step twice. Usually, a compiler optimization called **common subexpression elimination** would kick in so that *step was **only evaluated once**, and the value **reused for the second occurrence**. However, the compiler **can't use this optimization** here.

VTUADDA

The pointers timer1 and step might alias one another. In other words, the compiler cannot be sure that the write to timer1 doesn't affect the read from step.

In this case the second value of *step is different from the first and has the value *timer1. This forces the compiler to insert an extra load instruction.

The same problem occurs if you use structure accesses rather than direct pointer access. The following code also compiles inefficiently:

```
typedef struct {int step;} State;
typedef struct {int timer1, timer2;} Timers;
void timers_v2(State *state, Timers *timers)
{
timers->timer1 += state->step;
timers->timer2 += state->step;
}
```

The compiler evaluates state->step twice in case state->step and timers->timer1 are at the same memory address. The fix is easy: **Create a new local variable to hold the value of state->step so the compiler only performs a single load**.

In the code for timers_v3 we use a **local variable step to hold the value of state->step**. Now the compiler does not need to worry that state may alias with timers.

```
void timers_v3(State *state, Timers *timers)
{
int step = state->step;
timers->timer1 += step;
timers->timer2 += step;
}
```

VTUADDA

You must also be careful of other, less obvious situations where aliasing may occur. When you call another function, this function may alter the state of memory and so change the values of any expressions involving memory reads. The compiler will evaluate the expressions again.

Another pitfall is to **take the address of a local variable.** Once you do this, the variable is referenced by a pointer and so aliasing can occur with other pointers. The compiler is likely to keep reading the variable from the stack in case aliasing occurs. Consider the following example, which reads and then checksums a data packet:

```
int checksum_next_packet(void)
{
int *data; int N, sum=0;
data = get_next_packet(&N);
do
{
sum += *(data++);
} while (--N);
return sum;
}
```

Here get_next_packet is a function returning the address and size of the next data packet.

The previous code compiles to

checksum_next_packet

```
        STMFD r13!,{r4,r14}   ; save r4, lr on the stack
        SUB     r13,r13,#8      ; create two stacked variables
        ADD     r0,r13,#4       ; r0 = &N, N stacked
        MOV     r4,#0   ; sum = 0
        BL      get_next_packet         ; r0 = data
checksum_loop
        LDR     r1,[r0],#4      ; r1 = *(data++)
        ADD     r4,r1,r4; sum += r1
        LDR     r1,[r13,#4]     ; r1 = N (read from stack)
        SUBS    r1,r1,#1        ; r1-- & set flags
        STR     r1,[r13,#4]     ;N= r1 (write to stack)
        BNE     checksum_loop           ; if (N!=0) goto loop
        MOV     r0,r4   ; r0 = sum
        ADD     r13,r13,#8      ; delete stacked variables
        LDMFD r13!,{r4,pc}    ; return r0
```

Note how the compiler reads and writes N from the stack for every N--. Once you take the address of N and pass it to get_next_packet, the compiler needs to worry about aliasing because the pointers data and &N may alias. To avoid this, don't take the address of local variables. If you must do this, then copy the value into another local variable before use.

SUMMARY: Avoiding Pointer Aliasing

- Do not rely on the compiler to eliminate common subexpressions involving memory accesses. Instead create new local variables to hold the expression. This ensures the expression is evaluated only once.
- Avoid taking the address of local variables. The variable may be inefficient to access from then on.

3.7 Portability Issues

Here is a summary of the issues you may encounter when porting C code to the ARM.

• ***The char type.*** On the ARM, char is unsigned rather than signed as for many other processors. A common problem concerns loops that use a char loop counter i and the continuation condition $i \geq 0$, they become infinite loops. In this situation, armcc produces a warning of unsigned comparison with zero. You should either use a compiler option to make char signed or change loop counters to type int.

• ***The int type.*** Some older architectures use a 16-bit int, which may cause problems when moving to ARM's 32-bit int type although this is rare nowadays. Note that expressions are promoted to an int type before evaluation. Therefore, if i = -0x1000, the expression i == 0xF000 is true on a 16-bit machine but false on a 32- bit machine.

• ***Unaligned data pointers.*** Some processors support the loading of short and int typed values from unaligned addresses. A C program may manipulate pointers directly so that they become

unaligned, for example, by casting a char * to an int *. ARM architectures up to ARMv5TE do not support unaligned pointers. To detect them, run the program on an ARM with an alignment checking trap. For example, you can configure the ARM720T to data abort on an unaligned access.

• ***Endian assumptions.*** C code may make assumptions about the endianness of a memory system, for example, by casting a char * to an int *. If you configure the ARM for the same endianness the code is expecting, then there is no issue. Otherwise, you must remove endian-dependent code sequences and replace them by endian-independent ones.

• ***Function prototyping.*** The armcc compiler passes arguments narrow, that is, reduced to the range of the argument type. If functions are not prototyped correctly, then the function may return the wrong answer. Other compilers that pass arguments wide may give the correct answer even if the function prototype is incorrect. Always use ANSI prototypes.

• ***Use of bit-fields.*** The layout of bits within a bit-field is implementation and endian dependent. If C code assumes that bits are laid out in a certain order, then the code is not portable.

• ***Use of enumerations.*** Although enum is portable, different compilers allocate different numbers of bytes to an enum. The gcc compiler will always allocate four bytes to an enum type. The armcc compiler will only allocate one byte if the enum takes only eight-bit values. Therefore, you can't cross-link code and libraries between different compilers if you use enums in an API structure.

• ***Inline assembly.*** Using inline assembly in C code reduces portability between architectures. You should separate any inline assembly into small inlined functions that can easily be replaced. It is also useful to supply reference, plain C implementations of these functions that can be used on other architectures, where this is possible.

• ***The volatile keyword.*** Use the volatile keyword on the type definitions of ARM memory-mapped peripheral locations. This keyword prevents the compiler from optimizing away the memory access. It also ensures that the compiler generates a data access of the correct type. For example, if you define a memory location as a volatile short type, then the compiler will access it using 16-bit load and store instructions LDRSH and STRH

Summary

By writing C routines in a certain style, you can help the C compiler to generate faster ARM code. Performance-critical applications often contain a few routines that dominate the performance profile; concentrate on rewriting these routines using the guidelines of this chapter.

Here are the key performance points we covered:

- Use the `signed` and `unsigned int` types for local variables, function arguments, and return values. This avoids casts and uses the ARM's native 32-bit data processing instructions efficiently.
- The most efficient form of loop is a `do-while` loop that counts down to zero.
- Unroll important loops to reduce the loop overhead.
- Do not rely on the compiler to optimize away repeated memory accesses. Pointer aliasing often prevents this.
- Try to limit functions to four arguments. Functions are faster to call if their arguments are held in registers.

Learning Resources

Textbooks

1. Andrew N Sloss, Dominic Symes and Chris Wright, ARM system developers guide, Elsevier, Morgan Kaufman publishers, 2008.

Reference Books

1. Raghunandan. G.H, Microcontroller (ARM) and Embedded System, Cengage learning Publication,2019

2. The Insider's Guide to the ARM7 Based Microcontrollers, Hitex Ltd.,1st edition, 2005.

VTU Question paper questions

June/July 2024 (Same scheme)

Module – 3					
Q.5	a.	How registers are allocated to optimize the program? Develop an assembly level program to find the sum of first to integer numbers.	10	L2	CO3
	b.	How complier handles a "for loop" with variable number of iterations N and loop controlling with an example.	10	L2	CO3
OR					
Q.6	a.	Explain the following terms with an appropriate example : i) Pointer Aliasing ii) Portability issues.	10	L2	CO3
	b.	How function calling is efficiently used by ARM through APCS with an example program.	10	L2	CO3

June/July 2023 (2021 scheme)

Module-3

5 a. Write a C program that prints the square of the integers between 0 to 9 using functions and explain how to convert this C function to an assembly function with command. (10 Marks)

b. Discuss how instruction is scheduled in ARM. (10 Marks)

OR

6 a. Explain code optimization, profiling and cycle counting. (10 Marks)

b. Discuss how Registers are allocated to optimize the program. (10 Marks)